

# SOLID Study Guide

One-page refresher per principle, with examples from SOLID Race Lab  
Ingeniería de Software II · Universidad Nacional de Colombia · 2026-2

SOLID is a set of five design guidelines for object-oriented code, first named by Robert C. Martin. None of them is a law of nature — each is a rule of thumb that trades a little more structure now for a lot less pain later, when requirements change (and they always do). The examples below use the same vehicle-race code you work with in the workshop.

## S — SRP: Single Responsibility Principle

**Definition.** A class should have only one reason to change — that is, it should be responsible for one job, owned by one actor or concern.

**Why it matters.** When a class mixes unrelated jobs, a change to one of them (a new log format, a new print style) risks breaking the others, and the class becomes harder to read, test and reuse. Two people needing to change the same file for unrelated reasons is a classic symptom.

### *Before — Car mixes three responsibilities*

```
class Car:
    def move(self):
        self.position += self.speed
        print(f"{self.name} at {self.position}m") # rendering
        with open("log.txt", "a") as f:          # persistence
            f.write(f"{self.name}={self.position}\n")
```

### *After — each class has exactly one job*

```
class Car:
    def move(self):
        self.position += self.speed

class RaceLogger:
    def record(self, tick, racers): ...
    def save(self, path): ...
```

### Ask yourself:

- If I describe this class in one sentence, does the word “and” show up?
- Who would ask me to change this class — and is it more than one kind of person or concern?

## O — OCP: Open/Closed Principle

**Definition.** Software entities should be open for extension but closed for modification: you should be able to add new behavior without editing existing, already-tested code.

**Why it matters.** Code that must be edited every time a new case appears (a new vehicle type, a new profession) tends to accumulate if/elif chains, and every edit risks breaking a case that used to work. Polymorphism lets you add a case as new code instead.

### *Before — adding a vehicle means editing this function*

```
def speed_of(vehicle_type):
    if vehicle_type == "car":
        return 4
    elif vehicle_type == "truck":
        return 2
    # every new vehicle type means editing this function again
```

### *After — adding a vehicle means adding a class*

```
class Vehicle:
    def move(self): raise NotImplementedError

class Car(Vehicle):
    def move(self): self.position += 4

class Motorcycle(Vehicle):    # new, nothing above changed
    def move(self): self.position += random_step()
```

### **Ask yourself:**

- To support a new case, do I edit an existing class, or add a new one?
- Does the code that runs the race (Track.run) need to know each vehicle type by name?

## L — LSP: Liskov Substitution Principle

**Definition.** Objects of a subclass should be replaceable with objects of their base class without altering the correctness of the program — a subclass must honor the contract (pre/post-conditions, invariants) that callers rely on.

**Why it matters.** If a subclass secretly does less (or something different) than its base class promises, every piece of code written against the base class is now at risk, even though it was never told about the subclass. This is what makes LSP violations so easy to miss in code review and so painful in production.

### ***Before — UnreliableCar breaks the Vehicle contract***

```
class Vehicle:
    def move(self): ...    # contract: position never decreases, never raises

class UnreliableCar(Vehicle):
    def move(self):
        if random() < 0.15:
            raise RuntimeError("broke down!")    # breaks the contract
        self.position -= 3                        # breaks it too
```

### ***After — still unreliable. still substitutable***

```
class UnreliableCar(Vehicle):
    def move(self):
        if random() < 0.3:
            return    # stalls, but never breaks the contract
        self.position += 5
```

### **Ask yourself:**

- What does the base class promise callers, implicitly or explicitly?
- Could I swap this subclass in anywhere the base class is expected, and have nothing else notice?

# I — ISP: Interface Segregation Principle

**Definition.** Clients should not be forced to depend on methods they do not use. Prefer several small, specific interfaces over one large, general-purpose one.

**Why it matters.** A fat interface forces every implementer to either provide a real implementation for methods that make no sense for it, or fake one (raise `NotImplementedError`, return a dummy value) — both are code smells that spread confusion and fragile stub methods across the codebase.

## *Before — one interface, forced stub methods*

```
class VehicleActions(ABC):
    def move(self): ...
    def refuel(self): ...
    def fly(self): ...
    def pedal_harder(self): ...

class Bicycle(VehicleActions):
    def fly(self):
        raise NotImplementedError("Bicycles can't fly!")
```

## *After — small interfaces, only what applies*

```
class Movable(ABC):
    def move(self): ...

class Pedalable(ABC):
    def pedal_harder(self): ...

class Bicycle(Movable, Pedalable):
    def move(self): ...
    def pedal_harder(self): ...
```

## **Ask yourself:**

- Does every implementer of this interface have a real, meaningful implementation for every one of its methods?
- If I split this interface in two, would any class need to implement both?

## D — DIP: Dependency Inversion Principle

**Definition.** High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

**Why it matters.** When a high-level policy (“how to run a race”) is hardwired to specific low-level classes (“SportsCar, DeliveryVan”), it cannot be reused or tested without those exact classes. Depending on an abstraction (anything with move()/position/symbol) instead lets you inject whatever concrete objects you want — real ones, or lightweight test doubles.

### ***Before — Race builds its own concrete vehicles***

```
class Race:
    def __init__(self):
        self.vehicles = [SportsCar("Flash"),
                        DeliveryVan("Steady Eddie")]

    def start(self):
        return self.track.run(self.vehicles)
```

### ***After — Race depends on an abstraction. injected***

```
class Race:
    def __init__(self, racers, track=None):
        self.racers = racers          # any Racer-shaped object
        self.track = track or Track(length=30)

    def start(self):
        return self.track.run(self.racers)
```

### **Ask yourself:**

- Does this class construct its own collaborators, or receive them?
- Could I unit-test this class with a fake, in-memory version of its dependency, with no changes to the class itself?